

What “Compiled” Actually Means

Strings, casting, and a file that fails to exist

Week 2, Day 2 · Thursday, September 3 · SC 333 · 3:00–4:15

What you leave with today

- Why `(int) 3.9` is 3 but `"%.0f"` of 3.9 is 4 — **chop versus round**
- The seven String methods that carry you through Skill Builder 1
- `.java` → `.class` → a running program, watched in a terminal
- Why Java catches `int x = "hi";` **before your program runs**, and Python cannot

No quiz today. Q1 moved to **Tuesday, September 8** — which is after Recitation 2 is due, so every question on it comes from a lab you have already finished. Handout out, pen in hand.

Where we have been

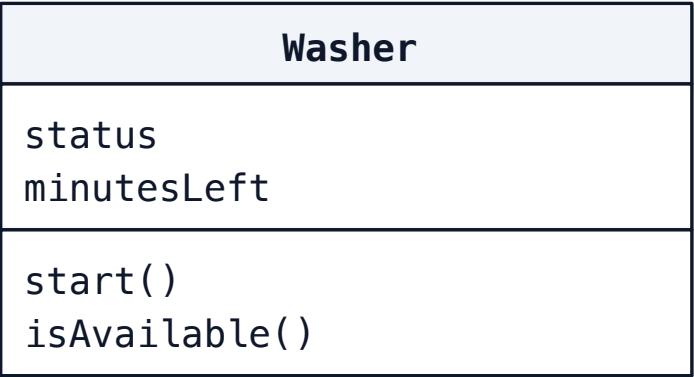
week 1 and Tuesday, on one screen each — then we pick up where we stopped

An object is three things

	Identity	State	Behavior
A marker	<i>this</i> one, not the other eleven in the box	colour · ink left · cap on	writes · dries out
A Robocode tank	its name in the battle list	energy · position · heading · gun heat	drives · scans · fires
A String	the object the arrow points at	the characters in it	length() charAt() substring()

Week 1, two identical markers. Same colour, same ink — same state, different identity, and I said Java has a different tool for each question and would not name them. **It is == and .equals(), and they are twenty minutes away.**

A UML class box is the same three compartments



The compartment

the name

the middle

the bottom

What it holds

what **kind** of thing this is

state — the fields every one of them has

behavior — what it can do

The diagram describes the class. Identity belongs to the object. One Washer diagram, forty washers in the building — each with its own state, and each one a *specific* washer. That distinction is week 3, and it starts here.

Tuesday: what a variable actually holds

PRIMITIVE

```
int n = 5;
```



the box holds **the value itself**

REFERENCE

```
String s = "hi";
```

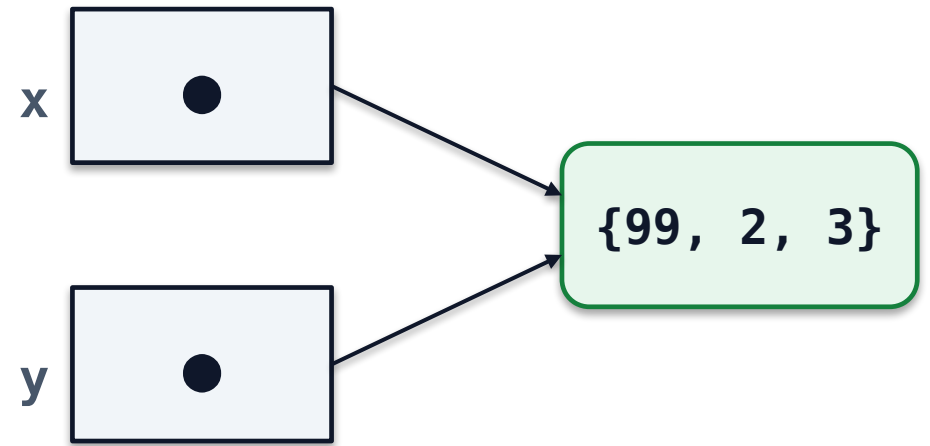


the box holds **an arrow**. The object lives elsewhere

Eight primitives, lowercase. Everything else — `String`, `int[]`, `Scanner`, every class you write — is the right-hand picture.

Copy a reference and you copy the arrow

```
int[] x = {1, 2, 3};  
int[] y = x;  
y[0] = 99;  
System.out.println(x[0]);    // 99
```



One array. Two names for it. Nothing was copied except the arrow — and that is why `==` is a question about *which object*, not about what is written in it.

Finish Tuesday

I said I would pick this up — so here it is

TimeDilation.java — every Tuesday idea in one file

In the file

```
final double speedOfLight = 2.99792458e8;
```

every variable declared with **a type first**

```
Math.sqrt(...)
```

 inside a double expression

```
printf("%1$.2f", ...)
```

Tuesday's idea

```
final
```

, and scientific notation is a double

static typing

the whole expression stays double

formatting — **which is today**

Run it with 0.9 and 10. It prints 22.94. Four things to notice and no more — this is a recap, not a re-teach.



Ten years for them. Twenty-three for you.

That is double arithmetic doing something you would **not** want an `int` doing.

And that `printf` on the last line is the rest of the period.

== versus .equals()

six minutes, board only — the one idea from Tuesday that decays fastest

Three strings. Three questions.

```
String a = "hi";  
String b = new String("hi");  
String c = "hi";
```

Hands up for each one. `a == b` · `a.equals(b)` · `a == c`. I want to see the split before I draw anything.

The third row is the dangerous one

Ask	Answer	Why
<code>a == b</code>	false	two objects, same letters
<code>a.equals(b)</code>	true	<code>.equals()</code> asks about contents
<code>a == c</code>	true	both literals — one pooled object, two names

R3 and R4 have both done Recitation 2 exercise 4 by now. What did it feel like when the tests told you two identical strings were not equal?



== will keep looking like it works.

Right up until the string comes from a Scanner, a file, or the network.

Then it is silently false, and **nothing crashes.**

== for primitives. .equals() for everything else.

Strings

Tuesday: the variable holds an arrow. Today: what is at the other end

Predict — what prints?

```
String s = "hello";  
s.toUpperCase();  
System.out.println(s);
```

Line 1 of the prediction table on your handout. **Write it down before I run it.**

Nothing happened. Say why.

```
String s = "hello";  
s.toUpperCase();  
System.out.println(s);    // hello
```

```
s = s.toUpperCase();  
System.out.println(s);    // HELLO
```

One character of difference: `s =`. The method never touched `s` — it built a new `String` and handed it back, and the first version dropped it on the floor.



A String cannot be changed.

Every method that looks like it edits one actually builds a **new** one and hands it back.

If you throw away the answer, you did nothing.

The method set worth knowing

Java	Python	Note
<code>s.length()</code>	<code>len(s)</code>	a method — it has parentheses
<code>s.charAt(i)</code>	<code>s[i]</code>	returns a char, which is a number
<code>s.substring(a, b)</code>	<code>s[a:b]</code>	stops before b, same as a slice
<code>s.equals(t)</code>	<code>s == t</code>	use this, not ==
<code>s.toUpperCase()</code>	<code>s.upper()</code>	returns a new String
<code>s.trim()</code>	<code>s.strip()</code>	returns a new String
<code>s.contains(t)</code>	<code>t in s</code>	

That is all you need this semester, and most of what SB1 needs. It is on your handout — do not copy it, listen instead.



The last character is `s.charAt(s.length() - 1).`

There is **no negative indexing**. No `s[-1]`, no `s[:-1]`.

You will write that expression for the rest of the course.

Get used to it now and it stops costing you anything.

Chop versus round

the most directly useful fifteen minutes of the week — SB1 needs both

Two ways to get fewer decimals. They do not agree.

```
double v = 3.999;  
  
(int) v           // 3  
String.format("%.0f", v) // "4"
```

Same number. Two answers. **Neither one is a bug** — they are two different operations and Skill Builder 1 asks for both, in the same program.



Casting truncates. Formatting rounds.

Choose on purpose.

Copy this line. It is where people lose points on SB1.

The two-decimal trick – SB1 needs this by name

```
double v = 1.4815297665908702;  
  
double t = ((int)(v * 100)) / 100.0;
```

Step	Value
$v * 100$	148.15...
$(\text{int})(v * 100)$	148
$\dots / 100.0$	1.48

Multiply up, chop, divide back down. Three steps, and the chop in the middle is the only one doing any work.

What if I write / 100?

```
((int)(v * 100)) / 100.0    // 1.48
((int)(v * 100)) / 100      // 1
```

148 / 100 is 1. **Both operands are int, so the answer is an int** — Tuesday's integer division and Recitation 2 exercise 1, arriving in a place you were not expecting them.

printf versus String.format

```
System.out.printf("%.2f%n", v);           // prints. returns nothing
String out = String.format("%.2f", v);    // returns the text
```

Same format string. One prints, one hands you the text. **SB1 needs both, in different places** — and a method that has to return a sentence cannot use the one that prints.

Recitation 2, exercise 7 — `spiceReport` builds SB1's output sentence almost exactly. Open it when you get stuck this weekend.

What “compiled” actually means

this part happens in a terminal — the slides are only the transcript

Hello.java — typed live, four lines

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("hi");  
    }  
}
```

Nothing on this slide is new. Recitation 0 took `public static void main` apart keyword by keyword. What happens to this file next is the new part.

A file you did not write

```
$ ls
Hello.java
$ javac Hello.java
$ ls
Hello.java  Hello.class
            ^ where did that come from?
```

You did not write it. `javac` **did** — and it is not for you to read.

The best four bytes in the course

```
$ xxd Hello.class | head -4
00000000: cafe babe 0000 0041 001d 0a00 0200 0307  ....A.....
      ^^^^  ^^^^
```

CAFEBABE. Every Java class file ever compiled starts with those four bytes — it is how a program knows it is looking at Java bytecode and not a JPEG. Gosling's team picked it in 1991 and it has never changed.

The rest is not for you. It is instructions for the JVM.

Handing the JVM a class name

```
$ javap -c Hello
0: getstatic      #7      // Field java/lang/System.out
3: ldc            #13      // String hi
5: invokevirtual #15      // Method println
8: return
$ java Hello
hi
```

java Hello — **no** .java, **no** .class. You hand the JVM a class name and it goes looking for an entry point inside it: `public` so it can call it, `static` so it can call it without constructing anything, `main` because that is the name it searches for.

Now watch it not exist

```
int x = "hi";
```

← add this line to main()

```
$ javac Hello.java
```

```
Hello.java:3: error: incompatible types:
```

```
    String cannot be converted to int
```

```
$ ls
```

```
Hello.java
```

^ no Hello.class. At all.

Nothing runs. There is no program to run.



In Python that line is a runtime error.

It happens when execution reaches it — which might be in production, on a Friday.

In Java there is **no program at all** until the types agree.

That is what you are buying with all the extra typing.

Two terms, then we move on

	When	What you get
Compile time	javac runs	type errors, missing methods, access violations
Run time	java runs	nulls, bad input, division by zero

Maven and IntelliJ do all of this for you. I am showing it once so you know what the green triangle is actually doing.

TimeUtils

two minutes, with the person next to you

3725 seconds → 01:02:05

```
public static String formatTime(int totalSeconds) {  
    int hours    = totalSeconds ____ 3600;  
    int rem      = totalSeconds ____ 3600;  
    int minutes  = rem ____ 60;  
    int seconds  = rem ____ 60;  
    return String.format("%02d:%02d:%02d", hours, minutes, seconds);  
}
```

Two operators, four blanks. **On your handout, with your neighbour.**

The answer — and why it needs `int`

```
int hours    = totalSeconds / 3600;  
int rem      = totalSeconds % 3600;  
int minutes  = rem / 60;  
int seconds  = rem % 60;
```

3725 / 3600 is 1, not 1.03. **If that had been a double this method would not work.** Integer division is not a wart you are working around — here it is exactly the tool.

`%02d` means *pad to two digits with zeros* — which is why you get `01:02:05` and not `1:2:5`.

Exit ticket — hand it in on your way out

- `(int) 2.99` and `String.format("%.0f", 2.99)` — **give both answers**
- You run `javac` and get an error. **How many .class files were produced?**
- One thing from Tuesday that makes more sense now than it did on Tuesday

Tear-off page at the back of your handout. **Three minutes.** Name and section on it — I read these tonight.

Before you go

- **Q1 — Tuesday Sep 8**, first ten minutes, on paper, no notes. Data types, operators, strings
- **Recitation 2** — finish at home, push, open the PR — **Sun Sep 6, 11:59 PM**
- **Drills** — Asterisks · Show me the Numbers — **Sun Sep 6**
- **Skill Builder 1 — Sun Sep 13**. Today was its content: casting, `printf`, `String.format`
- Reading — Think Java **Ch 2**, **Ch 3**, plus **§6.10** and **§9.3**
- **R3: no section Monday — Labor Day**. Your next one is **Sep 14**, and SB1 is due **Sep 13**

R3 — plan for that gap now. Office hours, tutoring, and Recitation 2 exercises 6–8, which are SB1's spine.

**Casting truncates. Formatting rounds.
Choose on purpose.**

and there is no program at all until the types agree